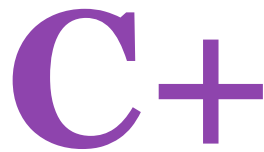


Momus Speaks

An Unflinching Critique

Spring Web Service Sandbox



Project: spring-web-service-sandbox

Stack: Java 25 · Spring Boot 4.0.3 · Hexagonal Architecture

Date: 9 March 2026

"No participation trophies."

1. The Premise

This is a sandbox project. By its own admission, it exists to “experiment with Spring Boot features.” The feature checklist reads like a Spring Boot greatest-hits compilation: REST API, JPA, Flyway, RabbitMQ, Testcontainers, Prometheus, Grafana, OpenAPI, hexagonal architecture. It is a resume project, and it looks like one.

The domain — books with four string fields — is the thinnest possible veneer over a database row. There is no business logic anywhere. The `BookService` is a pure pass-through: every method delegates directly to a port with zero transformation, zero validation, zero domain rules. The “domain” is an empty room decorated with expensive wallpaper.

As a sandbox for demonstrating infrastructure wiring and architectural scaffolding, it serves its purpose. But it would be dishonest to pretend the architecture is justified by the complexity of the problem.

2. Architecture Honesty

The hexagonal architecture here is architecture astronautics. Full stop.

The entire “domain” layer consists of:

- `Book` — a record with five fields
- `CreateBookCommand` — a record with four fields
- `BookUseCase` — four methods, all CRUD
- `BookStore` — three methods, all CRUD
- `BookImportPort` — one method
- `BookService` — five lines of delegation

Hexagonal architecture exists to protect domain logic from infrastructure concerns. **There is no domain logic to protect.** The `BookService` is a bureaucrat who receives a form, stamps it, and passes it to the next desk. The architecture has cost 17 source files for a feature set that could be delivered in 4. The cost has bought nothing.

The hexagonal boundary is already violated. `BookService.java` line 6 imports `org.springframework.stereotype.Service` — a Spring Framework annotation living inside

the “framework-free domain core.” The README states the domain has “no framework or infrastructure dependencies.” That is a lie.

3. Implementation Flaws

3.1 ISBN Validation is Wrong

CreateBookRequest.java:9

The regex `^\d{3}-\d{10}$` accepts `978-0132350884` (14 characters), but would reject valid ISBN-13s like `978-0-13-235088-4`. The test CSV data uses this same malformed format — tests validate the bug, not the requirement.

3.2 No Duplicate ISBN Protection

No unique constraint on `isbn` in the database schema. You can import the same CSV ten times and get ten copies of every book. For a domain where ISBN is literally a unique identifier by definition, this undermines the credibility of the model.

3.3 No Update or Delete Operations

The API supports Create and Read. No Update. No Delete. Once you bulk-import 10,000 books, there is no way to fix a mistake short of direct SQL. For a sandbox demonstrating REST patterns, the absence of PUT/PATCH and DELETE is a significant gap.

3.4 The Import Endpoint Lies About Being Async

BookController.java:57-77

The endpoint returns `202 ACCEPTED`, implying asynchronous processing. But the controller synchronously reads every CSV row, validates each one, and synchronously publishes each to RabbitMQ. For a 10,000-row CSV, the client blocks until all 10,000 messages have been published. The 202 is only partially honest.

3.5 No Error Reporting for Bulk Import

Invalid rows are logged and skipped. The client receives a bare 202 with no body. No count of valid rows, skipped rows, or failure details. Complete absence of feedback.

3.6 RabbitMQ Publishing Without Exchange

`RabbitBookPublishAdapter.java:18` — Publishing directly to the default exchange with the queue name as routing key. This is a beginner pattern. A sandbox demonstrating RabbitMQ should use an explicit exchange and binding.

3.7 No Dead Letter Queue

When `BookImportListener.onMessage()` fails, the message will be requeued indefinitely. No dead letter exchange, no retry policy, no error handling whatsoever. A poisoned message will spin forever.

3.8 No Transactional Boundary on Import

No `@Transactional` annotation on `BookService.create()` or `BookRepositoryAdapter.save()`. Spring Data's `save()` has its own transaction, but the service layer has no explicit transactional boundary.

3.9 The `host.docker.internal` Assumption

`prometheus.yml` uses `host.docker.internal:8080`. This only works on Docker Desktop (macOS/Windows). Not on Linux.

3.10 Hardcoded Credentials Everywhere

`application.yaml` hardcodes `sandbox/sandbox` database credentials. `application-local.yaml` sets `show-values: always` for actuator env endpoint — anyone who hits `/actuator/env` sees every credential. Combined with wide-open actuator exposure, this is an open door.

3.11 `findAll()` Without Pagination

After bulk-importing 10,000 books, `GET /books` returns all 10,000 in a single response. No pagination, no streaming, no limit. This contradicts the careful thought given to lazy CSV parsing.

3.12 Lombok in the Domain Model

`BookService.java` uses `@Slf4j` and `@RequiredArgsConstructor` from Lombok. Combined with `@Service`, the “clean domain” claim is thoroughly undermined. The domain has three framework dependencies: Spring, Lombok, and SLF4J.

4. What Is Missing

- **No integration test for the full import pipeline.** The most interesting feature has no end-to-end test. `SandboxApplicationTests` only verifies context loading.
- **No test for `BookRepositoryAdapter`.** The mapping logic (`toBook` , `toEntity`) is untested.
- **No DELETE or PUT endpoint.** A hiring manager absolutely notices this.
- **No consistent error response model.** `GlobalExceptionHandler` returns `Map<String, String>`. No error envelope.
- **No API versioning.**
- **No pagination.**
- **No CI/CD pipeline.**
- **No Dockerfile for the application.**

5. What Is Over-Engineered

- **The hexagonal architecture.** Seventeen files and six packages for a controller, a service that does nothing, a repository adapter, and a JPA repository.

- **The `CreateBookCommand` / `CreateBookRequest` distinction.** Structurally identical records with four identical String fields. The controller manually maps one to the other. Ceremony for ceremony's sake.

Begrudging respect: The Grafana dashboard provisioning. A pre-provisioned Grafana dashboard for monitoring the import listener of a sandbox CRUD app is delightful overengineering. It is also the most creative part of the project.

Retracted criticism: The `Validator` injection in `BookController`. Manual validation for non-`@RequestBody` inputs from CSV is the correct approach. One of the better decisions in the codebase.

6. The Verdict

Grade: C+

This is a competent but unambitious sandbox. It demonstrates that the author can wire Spring Boot components together, configure Docker infrastructure, and follow a hexagonal package structure. It does not demonstrate that the author understands *why* hexagonal architecture exists, because the domain is empty.

The code is clean, consistently formatted, and well-organized. The README is genuinely good. The test coverage exists at each architectural layer. The Grafana/Prometheus setup shows initiative.

But the project fails at the one thing a sandbox should do above all else: demonstrate judgment.

Choosing hexagonal architecture for a domain with no behavior, building a bulk import pipeline with no error feedback, exposing `findAll()` without pagination after demonstrating awareness of memory concerns — these choices reveal a developer who knows the tools but has not yet internalized when and why to use them.

To Earn a B or Above

1. Add actual domain logic (ISBN uniqueness, book status lifecycle, validation rules in the domain not the DTO)
2. Fix the broken hexagonal boundary (remove `@Service` from the domain)
3. Add an integration test for the import pipeline
4. Implement pagination
5. Add a dead letter queue

These changes would transform this from “I can configure Spring Boot” to “I understand software design.”
